

Deep Learning: Project 4

Ghibli Market: LoRA Style-Tuning with Stable Diffusion 1.5

Hesham Abdalla Jan Kobiolka
hesham.abdalla@utn.de jan.kobiolka@utn.de

June 25, 2026

This is a group project for **2–5 members**, worth a total of **30 points**, and is due on **26.07.2026**. If any issues should arise, feel free to reach out via the forum or by email. For your submission, please bundle a single `.zip` or `.tar.gz` archive containing your solution files.



1 What You Receive

You are given a small Ghibli-style image dataset in the `style_imgs/` folder. **No training code is supplied — you will write everything yourself.** `style_imgs/` folder contains:

- `style_imgs/original/` — full-resolution PNG/JPG reference images.
- `style_imgs/512/` — 512×512 crops, ready to train.

2 Tasks

2.1 Task 1 — Style Token

Add a new style token `<sks>` to the tokenizer so that it can be referenced in text prompts.

2.2 Task 2 — Dual-Adapter LoRA Finetuning

Finetune **both** the UNet **and** the text encoder with LoRA so that the prompt

a busy market, in <sks> style

produces clearly Ghibli-like images.

2.3 Task 3 — Deliver Code and Weights

Deliver runnable code **and** the resulting LoRA weights (see Section 4).

3 Implementation Requirements

Libraries: `diffusers`, `peft`, `PyTorch 2+`, `safetensors`.

Training script — `code/train_lora.py` (or notebook)

- Saves exactly one file → `lora_out/pytorch_lora_weights.safetensors`.
- Accepts CLI arguments: `data_dir`, `instance_token`, `output_dir`, `hyper-param`, and `--overwrite`.

Evaluation script — `code/eval_lora.py`

- Loads base SD 1.5 and your adapter.
- Renders at least three images for the prompt above (baseline optional).
- **Do not** expose a “style strength” slider.

`README.md`

The README file should include team members names, clear install & run commands, and the expected GPU / runtime.

Suggested Hyper-parameters

You may change any of these.

Parameter	Value
MODEL_NAME	runwayml/stable-diffusion-v1-5
INSTANCE_TOKEN	<sks>
RESOLUTION	512
RANK	8
LR	1e-4
MAX_STEPS	800

Example Commands

```
# Train
python code/train_lora.py \
    --data_dir style_imgs/512 \
    --instance_token "<sks>" \
    --output_dir lora_out \
    --rank 8 \
    --max_steps 800 \
    --overwrite

# Evaluate
python code/eval_lora.py \
    --weights lora_out/pytorch_lora_weights.safetensors \
    --prompt "a busy market, in <sks> style" \
    --outdir samples
```

4 Deliverables

Submit a single ZIP / tar.gz containing:

- lora_out/pytorch_lora_weights.safetensors
- code/train_lora.py
- code/eval_lora.py
- samples/ (≥ 3 images with adapter; baseline optional)
- requirements.txt or environment.yml
- README.md
- report.pdf (≤ 2 pages)

5 Grading

The project is graded out of **30 points**.

Criterion	Points
Correct dual-adapter LoRA setup	9
Visual fidelity to reference style	9
Prompt controllability (<sks> token)	6
Code quality & reproducibility	6
Total	30

6 Guidelines

- No training code is supplied; implement the full pipeline yourself.
- Both the UNet and the text encoder must be adapted with LoRA.
- The evaluation script must not expose a style-strength slider.
- All code must run end-to-end on a fresh environment and reproduce the adapter.

7 Resources

- Diffusers docs — <https://huggingface.co/docs/diffusers>
- PEFT docs — <https://github.com/huggingface/peft>
- LoRA paper — Hu et al., 2021 <https://arxiv.org/pdf/2106.09685>